Training a Generally Curious Agent

FAHIM TAJWAR*, Carnegie Mellon University, USA
YIDING JIANG*, Carnegie Mellon University, USA
ABITHA THANKARAJ, Carnegie Mellon University, USA
SUMAITA SADIA RAHMAN, North Carolina State University, USA
J ZICO KOLTER, Carnegie Mellon University, USA
JEFF SCHNEIDER, Carnegie Mellon University, USA
RUSS SALAKHUTDINOV, Carnegie Mellon University, USA

Efficient exploration is essential for intelligent systems interacting with their environment, but existing language models often fall short in scenarios that require strategic information gathering. In this paper, we present PAPRIKA, a fine-tuning approach that enables language models to develop general decision-making capabilities that are not confined to particular environments. By training on synthetic interaction data from different tasks that require diverse strategies, PAPRIKA teaches models to explore and adapt their behavior on a new task based on environment feedback in-context without more gradient updates. Experimental results shows that models fine-tuned with PAPRIKA can effectively transfer their learned decision-making capabilities to entirely unseen tasks without additional training. Unlike traditional training, our approach's primary bottleneck lies in sampling useful interaction data instead of model updates. To improve samples efficiency, we propose a curriculum learning strategy that prioritizes sampling trajectories from tasks with high learning potenti. To improve samples efficiency, we propose a curriculum learning strategy that prioritizes sampling trajectories from tasks with high learning potential. These results suggest a promising path towards AI systems that can autonomously solve novel sequential decision-making problems that require interactions with the external world.

1 Introduction

Large Language Models (LLMs) are the backbone for autonomous agents, systems capable of achieving goals independently with minimal human supervision or intervention. Being interactive and exploratory is a crucial part of completing its objectives. However, two challenges limit these interactive capabilities. Firstly, naturally occuring tasks and data are unstructured and lack the structure needed to guide smooth model interactions. Second, directly deploying models into the real world to collect interaction data can produce critical errors.

Using *in-context learning* (ICL), LLMS can be trained on synthetic data, that allows them to adapt to tasks with minimal demonstrations[1]. The model can also be taught *in-context reinforcement learning*[2]. In this way, the model learns the general method to solve new problems, instead of being fed raw data. This paradigm shares similarities to supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF) stages of training a language model (vs pretraining) where only a relatively small number of examples is needed to produce a model that can generate responses to a wide range of queries that it is not trained on. This approach also closely follows the principles of *meta reinforcement learning* [3]. Another popular notion of motivation is *intrinsic motivation* [4]. Our work differs from this notion of curiosity in that we do not leverage any intrinsic motivation, rather we train our agents to explore and interact with an entirely unseen environment to gather required information that is needed for completing the task at hand. PAPRIKA can be thought of as a from of amortized exploration, since the main goal is to learn good exploration strategies from trajectories from many different environments on a new problem.

*Equal Contribution.

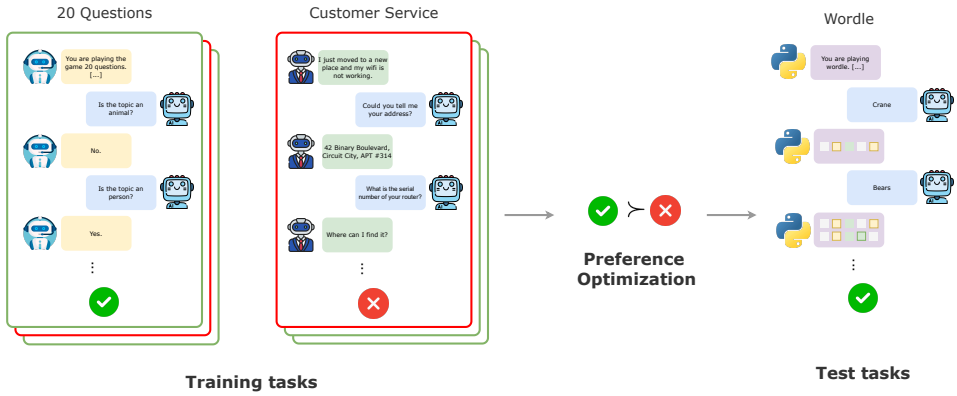

Fig. 1. **(Overview of PAPRIKA)**
A diverse set of tasks where an LLM agent needs strategic information gathering to succeed are designed, then an LLM is trained on self-generated data to prefer higher performing trajectories. The resulting behavior learned by PAPRIKA can transfer zero-shot to unseen tasks, showcasing its potential to build general decision making agents.

Using a base model, we generate interaction trajectories and assign scores based on their success in achieving the tasks' objectives. We then apply a sequential variant of Direct Preference Optimization[5] to increase the likelihood of successful trajectories. Our approach's primary bottleneck lies in sampling useful interaction data and not computational costs. As such, a curriculum learning strategy was introduced. We refer to the overall framework as PAPRIKA [1]. It highlights in-context RL using synthetic data rather than traditional costly task-specific training.

2 Related Works

LLM alignment. Alignment or post-training is a crucial step for creating a helpful LLM assistant. Typically, RLHF pipelines[6] work best. We focus on the multi-turn settings where the agent has to interact with an environment iteratively, similar to [7]. There are many existing environments and dataset focusing on multi-turn interactions, most of which focus on interactions with humans. Rather than a particular task, we focus on evaluating LLM's general ability to solve sequential decision making problems where the agent needs to explore and exploit.

In-context reinforcement learning. In-context learning (ICL) is the ability where LLMs can learn a new task from a small number of demonstrations without gradient updates[1]. Exsiting ICL focuses on single-turn interaction. We focus on in-context reinforcement learning [2, 8–10]. We mainly focus on diverse environments and study how well the decision making abilities generalize to completely new environments.

Curriculum learning in RL.. Curriculum learning[11] shows the data to the model in a non-uniform order, inspired by the fact that humans tend to learn things in a sequential order[12] and is particularly appealing for RL because learning easier tasks first could build scaffold toward solving difficult tasks that the agent could not solve otherwise[13–15]. Concurrent works[16] has studied curriculum learning extensively for training LLMs particularly to improve reasoning. Given grouping metadata, effective curriculum learning can be designed.

[1]The name is inspired by the movie "Paprika" (2006), where a dream detective navigates vast and strange dream worlds to solve different mysteries.

3 Preliminaries

Many decision making problems can be formalized as a partially observable Markov decision process (POMDP), and we assume each *task* τ to be a POMDP,

A *group* (or task group, used interchangeably),

$$G = \{\tau_1, \tau_2, \ldots, \tau_{|G|}\}$$

is a high-level grouping of tasks that share similar strategies. For example, the game twenty questions is a group, and guessing the word "`apple`" is one task in it. From the agent's perspective, each task is a black box that takes the agent's action a_t and outputs an observation o_t, both of which are strings.

An episode,

$$h = (o_0, a_0, \ldots, o_H, a_H)$$

is the agent's interaction trajectory within a single task, with $h_{p:q}$ denoting a slice of it similar to array slicing. An episode terminates when the agent achieves the objective or the maximum number of allowed interactions is reached, at which point the environment emits a single score $r(h)$.

Let π denote the LLM agent and $h \sim \pi \circ \tau$ denote sampling a trajectory from task τ using policy π. The performance of a policy on a group is

$$\texttt{Perf}(G) = \frac{1}{|G|} \sum_{\tau \in G} \mathbb{E}_{h \sim \pi \circ \tau}[r(h)]. \tag{1}$$

The agent is trained on a finite set of groups $\mathcal{G}_{\text{train}}$, and the goal is to perform well on unseen groups $\mathcal{G}_{\text{test}}$.

4 Method

Prior works have shown that LLMs can perform poorly even on the simple decision making task of multi-armed bandits[17], and that fine-tuning on synthetic trajectories generated by known algorithms such as UCB can teach them to do better[18]. However, this idea is limited in scope: we want LLMs to explore strategically in far more complex settings, for most tasks there is no known algorithm like UCB to generate good trajectories from, and it is infeasible to collect data for every task we care about. PAPRIKA solves these issues in steps. First, a suite of complex decision-making tasks requiring strategic information gathering is designed. Next, in the absence of known good algorithms, existing LLMs generate trajectories with better decision making behaviors through diversity-encouraging sampling. The LLM is then fine-tuned to prefer higher performing trajectories, in a fashion similar to STaR[19], and these behaviors often generalize to unseen task groups without additional training. Finally, a curriculum learning algorithm dynamically chooses which subset of tasks to train on next to improve data efficiency.

4.1 Task Design

The task groups should have four desired properties: **(1)** they are purely text based, **(2)** they require multi-turn interaction, where the agent has to understand prior history in its context and choose actions that maximize the probability of future success, **(3)** they are partially observable, so the agent must simultaneously explore to reveal more information and exploit to solve the task efficiently, and **(4)** they are diverse and require different strategies to succeed. With these requirements in mind, 10 task groups are designed, summarized in Table 1.

In all of them, an LLM agent solves a task through sequential interaction with the task-specific environment, which provides both intermediate observations and a task reward at the end of an episode.

Table 1. Summary of the task groups used by PAPRIKA.

Task Group	# Train Tasks	# Test Tasks	Maximum Turns	Env Feedback	Uses COT
Twenty questions	1499	367	20	LLM generated	✗
Guess my city	500	185	20	LLM generated	✗
Wordle	1515	800	6	Hardcoded program	✓
Cellular automata	1000	500	6	Hardcoded program	✓
Customer service	628	200	20	LLM generated	✗
Murder mystery	203	50	20	LLM generated	✗
Mastermind	1000	500	12	Hardcoded program	✓
Battleship	1000	200	20	Hardcoded program	✓
Minesweeper	1000	200	20	Hardcoded program	✓
Bandit best arm selection	81	1	21	Hardcoded program	✓

Following prior work[20], classic guessing games like *twenty questions* and *guess my city* are included, which require guessing a secret topic as quickly as possible by asking a sequence of questions. In *Wordle* and *Mastermind*, the agent needs to guess a secret 5-letter word and 4-digit code respectively, where the environment gives feedback about the similarity between the guess and the target, and the agent must refine its future guesses accordingly.

Customer service and *murder mystery* are dynamic text-based groups: an LLM plays the role of the environment and generates intermediate observations based on a given success criterion. *Cellular automata* simulates coding against interpreter feedback in a toy setting, where the agent makes inferences about the transition rule of a 1D elementary cellular automaton by observing inputs and outputs. *Minesweeper* and *Battleship* require interacting with 2D grids to find hidden items within a fixed number of turns.

Finally, a modified version of the multi-armed bandit group is included, where the agent works on the *bandit best arm selection* problem with the following distinctions: (1) we let the agent employ Chain of Thought Reasoning (CoT) before choosing arms so that they can transfer good strategies learned from the other tasks, (2) we let the agent interact with the task environment in a multiturn way, (3) and the agent has the ability to choose arms and observe rewards for a fixed number of turns and then measure its accuracy in deciding the arm with the highest reward, instead of reducing regret. This reduces computational cost over generating COTs for a large number of turns, since the difference in regret between different models is not meaningful when the number of turns is not large enough.

For tasks requiring general knowledge about the world, another LLM (typically GPT-4o-mini) is employed as the environment, while tasks with rule-based observations and rewards use hardcoded programs, which are more reliable than LLMs[21]. To prevent reward hacking, another LLM or a hardcoded program is used as a judge to filter out unsuccessful trajectories that got incorrectly labeled as successful. For task groups requiring complex reasoning, letting the agent think using chain-of-thought (COT) prompting[22] before generating a final answer improves performance significantly, similar to ReAct[23].

As an example of a task group with an LLM-simulated environment, Figure 2 shows the first user prompt received by the agent playing twenty questions, containing general instructions about the task and the type of hidden topic it needs to guess.

Twenty Questions Agent Prompt

You are playing a game of 20 Questions. Your goal is to guess the name of a thing or person by asking up to 20 yes-or-no questions. After each question, you will receive an answer: 'Yes' or 'No.' Use the answers provided to refine your guesses.
Here are your instructions:
- You can ask only yes-or-no questions.
- After receiving each answer, you should adapt your questions based on the new information.
- Your goal is to guess the topic in as few questions as possible.
- If you're confident, you can make a guess before reaching 20 questions.
The game starts now. You are trying to guess a clothing. Ask your first question!

Fig. 2. Example agent prompt for the twenty questions task group, where the environment is simulated by another LLM.

The environment here is another LLM that receives the secret topic and answers the agent's questions strictly with 'Yes', 'No', or 'Goal reached'. In contrast, task groups like wordle use a hardcoded program as the environment: Figure 3 shows the observation the program generates given the secret word "toast" and the agent's guess "boost".

Wordle Task Environment Feedback

First letter, b, is not in the target word
Second letter, o, is correct and in the correct position in the target word
Third letter, o, exists in the target word but in a different position
Fourth letter, s, is correct and in the correct position in the target word
Fifth letter, t, is correct and in the correct position in the target word
Make your next guess about the hidden word. Please try to be concise. Format your response in the following way: `<Think>` Any step-by-step, short and concise thinking to strategically determine the next guess for the secret word `</Think>`
`<Answer>` your guess of what the word should be `</Answer>`

Fig. 3. Example environment feedback for the wordle task group, generated by a hardcoded program.

4.2 Dataset Construction

The data generated from these task groups must be diverse, which allows the model to learn different strategies without the risk of overfitting. This is accomplished by generating a large number of trajectories at a high temperature with Min-p sampling[24]. Min-p sampling uses an adaptive threshold $p_{\text{scaled}} \propto p_{\text{max}}$, where p_{max} is the highest probability predicted by the model on the next token, and truncates the vocabulary to tokens with probability larger than p_{scaled} — this keeps the trajectories coherent even at higher temperatures. For each chosen task, n_{sample} trajectories are generated and a preference pair (h_w, h_l) is constructed, where h_w is the highest scoring trajectory (succeeds at the fewest number

of turns) and h_l is sampled randomly from the lower scoring trajectories instead of the worst one, in order to increase the diversity of the dataset. These act as proxies for desirable and undesirable behaviors. A dataset $\mathcal{D} = \left\{\left(h^w, h^l\right)^{(i)}\right\}_{i=1}^{N}$ is a collection of such trajectory pairs.

4.3 Optimization

Supervised fine-tuning. If the winning episodes are taken as expert behavior, the losing episodes can be discarded and the likelihood of the winning ones maximized:

$$\mathcal{L}_{\text{SFT}}(\mathcal{D}_{\text{SFT}}) = -\mathbb{E}_{\mathcal{D}_{\text{SFT}}}\left[\frac{1}{\sum_{t=0}^{|h_w|}|a_t^w|}\sum_{t=0}^{|h_w|}\log \pi_\theta\left(a_t^w \mid h_{:t}^w\right)\right] \tag{2}$$

where $\mathcal{D}_{\text{SFT}}$ is the dataset used for supervised fine-tuning and $|a|$ is the number of tokens of the agent response, discarding the environment generation.

Direct preference optimization. DPO[5] directly optimizes the Bradley-Terry model[25] for preferences. In this setting each trajectory consists of multiple rounds of interaction, so the original DPO objective does not apply, and a multi-turn version[7] is used instead:

$$\mathcal{L}_{\text{DPO}}\left(\mathcal{D}_{(DPO)}\right) = -\mathbb{E}_{\mathcal{D}_{\text{DPO}}}\left[\log \sigma\left(\sum_{t=0}^{|h^w|}\beta \log \frac{\pi_\theta\left(a_t^w|h_{:t}^w\right)}{\pi_{\text{ref}}\left(a_t^w|h_{:t}^w\right)} - \sum_{t=0}^{|h^t|}\beta \log \frac{\pi_\theta\left(a_t^l|h_{:t}^l\right)}{\pi_{\text{ref}}\left(a_t^l|h_{:t}^l\right)}\right)\right] \tag{3}$$

where $\mathcal{D}_{\text{DPO}}$ is the preference dataset, a_t^w and a_t^l are the action tokens generated by the model at turn t in the preferred and dispreferred trajectories h^w and h^l, and π_{ref} is the reference policy, taken to be the initial model. The main difference with standard DPO is that the loss is calculated only on the action tokens — the log probability ratios of the environment generated tokens are not included. DPO is chosen because it is less compute intensive and allows decoupling the data collection and policy improvement steps, though online RL with more resources would be expected to give even stronger results.

Combining objectives. DPO can have the unintended effect of reducing the probability of the preferred trajectories as well, known as unintentional unalignment[26]. The RPO objective[27] mitigates this issue by combining the SFT and DPO losses:

$$\mathcal{L}_{\text{RPO}}(\mathcal{D}_{\text{DPO}}) = \mathcal{L}_{\text{DPO}}(\mathcal{D}_{\text{DPO}}) + \alpha \mathcal{L}_{\text{SFT}}(\mathcal{D}_{\text{DPO}}) \tag{4}$$

where α is a hyper-parameter, set to 1.0 for the rest of the paper.

4.4 Scalable Online Learning Curriculum

It is relatively easy to design a large number of tasks, but harder to decide which ones to train on. If a task is too difficult for the current model, the generated trajectories carry no meaningful learning signal, and since generating trajectories is the dominant cost of training, tasks where the model can make meaningful progress should be prioritized — a form of curriculum learning[11]. The only way to know whether a task yields good learning signal is to actually perform rollouts on it, which is expensive, so an efficient curriculum must predict this without the rollout. A natural assumption is that similar tasks have similar levels of learning signal, and such groupings can be obtained through metadata or prior knowledge.

Algorithm 1 Task selection with UCB

Input: Number of arms K, number of samples C, number of rounds T, model π
Initialize: $s_k = 0$, $n_k = 0$, `Buffer`
for each round $t = 1, 2, \ldots, T$ **do**
  Compute $\theta_k = \frac{s_k}{n_k} + \sqrt{\frac{2 \log \sum_{k=1}^{K} n_k}{n_k}}$ for each k
  Select $k^\star = \arg\max_k \theta_k$
  Sample τ from group $k^\star$
  Sample C trajectories from τ and add to `Buffer`
  Compute an estimate for $\hat{v}_\pi(\tau)$ using Eq 5
  Update: $s_{k^\star} = s_{k^\star} + \hat{v}_\pi(\tau)$, $n_{k^\star} = n_{k^\star} + 1$
end for
Construct $\mathcal{D}$ from `Buffer` and train the model π

Measuring learning potential. The average performance of π on τ is $R_\pi(\tau) = \mathbb{E}_{h \sim \pi \circ \tau}[r(h)]$ and the variance is $\sigma_\pi^2(\tau) = \mathbb{E}_{h \sim \pi \circ \tau}\left[(r(h) - R_\pi(\tau))^2\right]$. Based on these, the learning potential of a single task is defined as

$$v_\pi(\tau) = \frac{\sqrt{\sigma_\pi^2(\tau)}}{R_\pi(\tau)}. \tag{5}$$

This quantity is known as the coefficient of variation in statistics, a dimensionless quantity that measures the population's variability relative to the mean. Variance naturally measures the possibility of getting diverse trajectories from sampling, which matters because DPO suffers when the preferred and dispreferred responses are not sufficiently different[28]. On the other hand, different tasks can have vastly different reward scales, and normalizing the standard deviation by the mean allows comparing different tasks directly.

Sampling tasks. Since it is infeasible to evaluate $v_\pi(\tau)$ for all tasks, tasks are sampled from each group instead. Given a collection of K groups, choosing which group to sample from next to maximize learning potential can be formulated as a multi-armed bandit problem, solved here with the Upper Confidence Bound (UCB) algorithm[29]. Each action corresponds to a group of tasks: one task is sampled uniformly from the chosen group, the model is evaluated on it with C rollouts, and these statistics update the mean estimate of that group. After a sufficient number of episodes are sampled, the dataset is constructed and the model is trained with the objectives above. Algorithm 1 shows the pseudocode.

5 Results

The empirical study answers the following research questions: **(1)** Can training on self-generated trajectories from a diverse range of task groups equip LLMs with sequential decision making capabilities that generalize to unseen task groups without training on them? **(2)** Can curriculum learning improve the data efficiency of this training mechanism? **(3)** Does PAPRIKA hurt the model's regular abilities?

Experimental setup. Llama-3.1-8B-Instruct[30] and Gemma-3-12B-IT[31] models are used for all experiments. Training data is generated with Min-p sampling at temperature 1.5 and Min-p parameter 0.3, which consistently produced diverse data resulting in higher test-time accuracy. For each task in the training split, $n_{\text{sample}} = 20$ trajectories are generated (100 for mastermind), which after filtering results in 17,181 training trajectories for supervised fine-tuning and 5,260 trajectory pairs for RPO over all task groups. The learning rate is 10^{-6} for supervised fine-tuning and 2×10^{-7} for

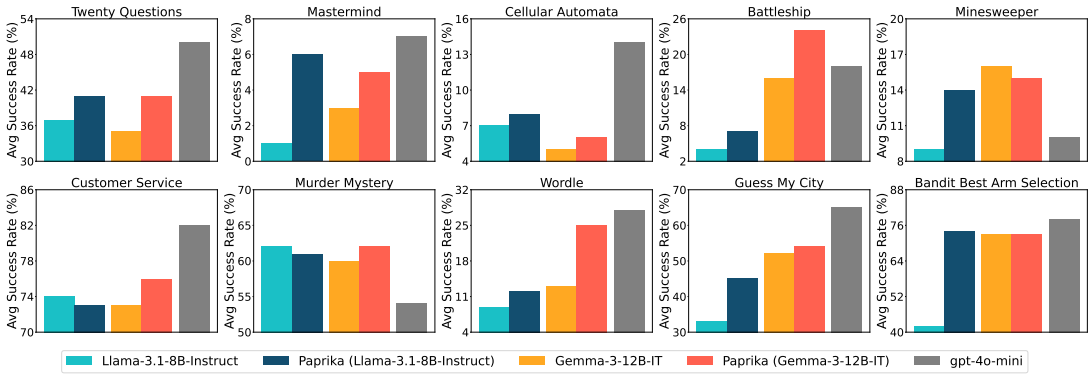

Fig. 4. **(PAPRIKA improves success rate on a diverse range of task groups)** Average success rate on all 10 task groups at temperature 0.7. PAPRIKA generally improves performance of both Llama-3.1-8B-Instruct and Gemma-3-12B-IT models.

RPO with batch size 32, using an AdamW optimizer[32] with a cosine annealing schedule and warmup ratio 0.04[33]. Supervised fine-tuning is always run first, followed by further fine-tuning with the RPO objective. During evaluation, 4 trajectories are generated for each task in the test set with Min-p parameter 0.3 at temperature 0.7, and the average success rate is reported. For task groups with hardcoded feedback, a failure to follow formatting instructions counts as a failure in the task.

PAPRIKA improves LLM decision making abilities. The toy group of bandit best arm selection requires strategic use of a fixed sampling budget to quickly discard unpromising arms. While previous work improved LLMs on bandits by training on trajectories from optimal bandit algorithms, PAPRIKA improves the average success rate of Llama-3.1-8B-Instruct on this group from 42.25% to 62.25% after only seeing trajectories from *other* task groups, without needing any optimal algorithm to generate synthetic data. On the full suite, Figure 4 shows that training on filtered trajectories from all 10 task groups improves the success rate of both models. Averaged across all groups, PAPRIKA increases the Llama model's performance by 47% of its original success rate after training on only about 22,500 trajectories.

PAPRIKA can teach LLMs generalizable strategies. To study whether the learned strategies zero-shot transfer to entirely different groups, a set of leave-one-out (LOO) experiments is performed: one group is chosen, the LLM is trained on trajectories from every other group, and the resulting model is tested on the left-out group. Additionally, for each group a model is trained and tested on that single group alone. Figure 5 shows the results: the LOO models can match or sometimes even exceed the performance of group-specific training, improving success rate on 9 out of 10 task groups compared to the initial model. Moreover, PAPRIKA trained on all 10 groups outperforms single-group training on 7 out of 10 groups, showing that the model learns better in-group strategies when it observes trajectories from other groups. The transfer is not universal — the improvement on mastermind is minimal and wordle shows negative transfer — but scaling up the number of task groups could keep improving zero-shot decision-making abilities.

Curriculum learning can improve data efficiency. The biggest bottleneck of PAPRIKA is the time required to generate a large number of trajectories, and harder tasks give smaller learning signal for an equal sampling budget. To test the curriculum, GPT-4o-mini classifies the tasks in twenty questions into 3 difficulty categories, resulting in 477 easy, 726

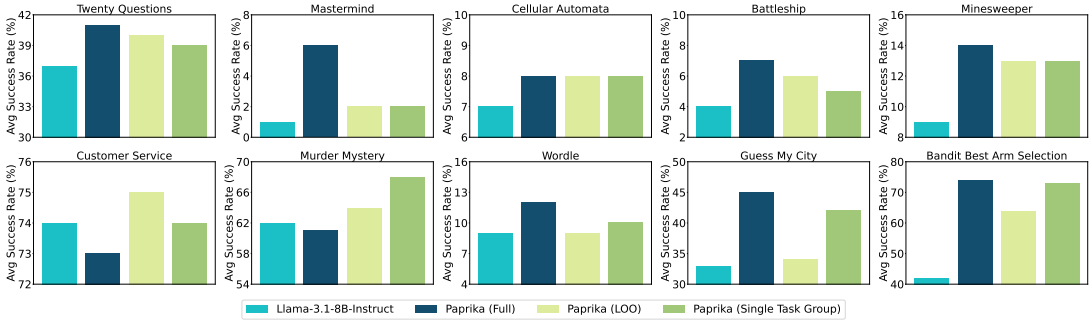

Fig. 5. **(Testing generalization of PAPRIKA via leave-one-out and single task group experiments)** PAPRIKA's zero-shot performance on unseen task groups is tested by leave-one-out (LOO) experiments, where the LLM is trained on every task group except the one it is tested on. PAPRIKA (Single Task Group) trains and tests the LLM on a single group. The learned decision making abilities often transfer well to new tasks without any additional training.

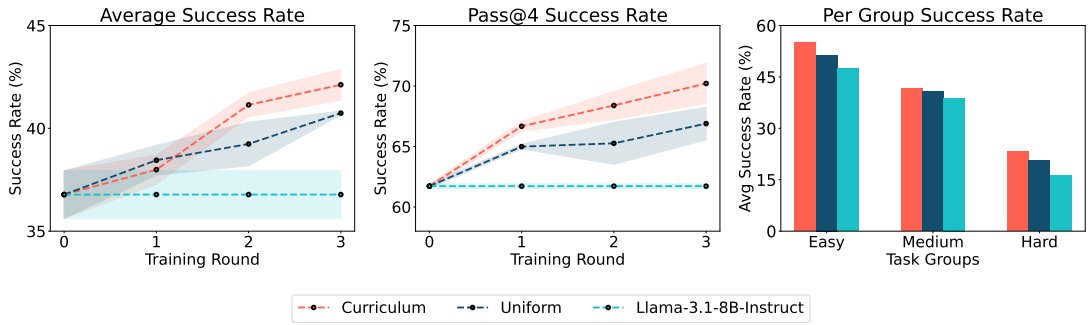

Fig. 6. **(Multi-round training with curriculum on twenty questions)** Performance of the curriculum learning algorithm against uniform sampling for multi-round training, with Llama-3.1-8B-Instruct as the initial model. (**Left**) Average success rate at each round. (**Middle**) Pass@4 success rate at each round. (**Right**) Success rate per each of easy, medium, and hard task groups.

medium, and 296 hard topics in the train split. The curriculum learning algorithm is run for 3 rounds on Llama-3.1-8B-Instruct: at each round, 250 tasks are sampled from the train set according to Algorithm 1, using the number of turns needed to solve a task as a proxy for reward in Eq 5. 20 trajectories are generated per task using the previous round's checkpoint, which is then trained on the resulting dataset. Figure 6 shows the results: after three rounds, the curriculum outperforms uniform random sampling of tasks by 1.4% and 3.3% at average and pass@4 accuracy respectively.

PAPRIKA does not hurt LLMs' regular capabilities. The tasks are designed so that an agent capable of better strategic exploration solves them faster, and PAPRIKA indeed reduces the average number of turns it takes the agent to solve tasks, implying that it chooses more optimal actions at intermediate steps. Finally, since scaling up PAPRIKA should not come at the cost of the model's general abilities, the fine-tuned model is compared against Llama-3.1-8B-Instruct on a set of standard benchmarks. Table 2 shows the findings: PAPRIKA does not result in any noticeable performance degradation.

Table 2. **(Evaluation of PAPRIKA on standard tasks)** Evaluation of PAPRIKA vs Llama-3.1-8B-Instruct on standard benchmarks (numbers in parenthesis represent standard error over 3 seeds). PAPRIKA does not result in significant model degradation.

Model	MT-Bench	AlpacaEval	GPQA	Math (Hard)	MMLU-Pro	IFEval
Llama-3.1-8B-Instruct	7.88	**33.6**	**33.5**	24.6	**46.7**	84.4
+ PAPRIKA	**8.14 (0.03)**	33.5 (0.3)	32.8 (1.5)	**25.3 (0.3)**	46.2 (0.1)	**85.4 (0.3)**

6 Discussion

This paper presented a scalable fine-tuning method to improve the multi-turn decision making abilities of LLMs, and showed that the strategies learned through it can generalize zero-shot to unseen tasks. The approach has a few limitations. First, since rejection sampling on self-generated data is used to teach the model better behaviors, the starting model needs to exhibit good behavior within a reasonable generation budget, so PAPRIKA would perform worse in the absence of a good base model. Second, offline preference tuning algorithms were used due to the lack of computational resources; running online RL on diverse tasks instead is a promising future direction given its recent success in other domains[21]. Third, the environments, despite being designed with the help of GPT-4o-mini, required a lot of human effort to implement, training an LLM to scalably generate suitable tasks is a new axis of improvement. Finally, the performance of the curriculum learning algorithm depends heavily on the quality of the task group clusters, which leaves room for further study. More broadly, this line of work can produce models with stronger strategic exploration and decision making capabilities, and while the experiments here are confined to relatively simple and controlled environments, it remains an open question what kind of impact truly agentic systems will have on society.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[2] Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, et al. In-context reinforcement learning with algorithm distillation. *arXiv preprint arXiv:2210.14215*, 2022.

[3] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A survey of meta-reinforcement learning. *arXiv preprint arXiv:2301.08028*, 2023.

[4] Jürgen Schmidhuber. Curious model-building control systems. In *Proc. international joint conference on neural networks*, pages 1458–1463, 1991.

[5] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[6] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[7] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to q^*: Your language model is secretly a q-function, 2024. URL https://arxiv.org/abs/2404.12358.

[8] Sharath Chandra Raparthy, Eric Hambro, Robert Kirk, Mikael Henaff, and Roberta Raileanu. Generalization to new sequential decision making tasks with in-context learning. *arXiv preprint arXiv:2312.03801*, 2023.

[9] Jonathan Lee, Annie Xie, Aldo Pacchiano, Yash Chandak, Chelsea Finn, Ofir Nachum, and Emma Brunskill. Supervised pretraining can learn in-context reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Licong Lin, Yu Bai, and Song Mei. Transformers as decision makers: Provable in-context reinforcement learning via supervised pretraining. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=yN4Wv17ss3.

[11] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.

[12] Burrhus F Skinner. Reinforcement today. *American Psychologist*, 13(3):94, 1958.

[13] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

[14] Carlos Florensa, David Held, Markus Wulfmeier, Michael Zhang, and Pieter Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on robot learning*, pages 482–495. PMLR, 2017.

[15] Meng Fang, Tianyi Zhou, Yali Du, Lei Han, and Zhengyou Zhang. Curriculum-guided hindsight experience replay. *Advances in neural information processing systems*, 32, 2019.
[16] Thomas Foster and Jakob Foerster. Learning to reason at the frontier of learnability, 2025. URL https://arxiv.org/abs/2502.12272.
[17] Akshay Krishnamurthy, Keegan Harris, Dylan J. Foster, Cyril Zhang, and Aleksandrs Slivkins. Can large language models explore in-context?, 2024. URL https://arxiv.org/abs/2403.15371.
[18] Allen Nie, Yi Su, Bo Chang, Jonathan N. Lee, Ed H. Chi, Quoc V. Le, and Minmin Chen. Evolve: Evaluating and optimizing llms for exploration, 2024. URL https://arxiv.org/abs/2410.06238.
[19] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022. URL https://arxiv.org/abs/2203.14465.
[20] Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. Lmrl gym: Benchmarks for multi-turn reinforcement learning with language models. *arXiv preprint arXiv:2311.18232*, 2023.
[21] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.
[22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.
[24] Minh Nguyen, Andrew Baker, Clement Neo, Allen Roush, Andreas Kirsch, and Ravid Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent llm outputs. *arXiv preprint arXiv:2407.01082*, 2024.
[25] Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4): 324–345, 1952.
[26] Noam Razin, Sadhika Malladi, Adithya Bhaskar, Danqi Chen, Sanjeev Arora, and Boris Hanin. Unintentional unalignment: Likelihood displacement in direct preference optimization, 2024. URL https://arxiv.org/abs/2410.08847.
[27] Richard Yuanzhe Pang, Weizhe Yuan, Kyunghyun Cho, He He, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization, 2024. URL https://arxiv.org/abs/2404.19733.
[28] Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddartha Naidu, and Colin White. Smaug: Fixing failure modes of preference optimisation with dpo-positive, 2024. URL https://arxiv.org/abs/2402.13228.
[29] Peter Auer. Using upper confidence bounds for online learning. In *Proceedings 41st annual symposium on foundations of computer science*, pages 270–279. IEEE, 2000.
[30] MetaAI, Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.
[31] Gemma-Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, et al. Gemma 3 technical report, 2025. URL https://arxiv.org/abs/2503.19786.
[32] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL https://arxiv.org/abs/1711.05101.
[33] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017. URL https://arxiv.org/abs/1608.03983.